

DOCUMENTACIÓN TÉCNICA

Portafolio Spotify

Pipeline ETL con Modelado Dimensional

Versión 1.0 · Mayo 2026

Pedro Parada · Technology Intern, JPMorgan Chase

Este documento describe la arquitectura, módulos y decisiones de diseño del pipeline ETL desarrollado para procesar el historial de reproducción de Spotify. El sistema implementa un esquema en estrella sobre Apache Parquet, con enriquecimiento geográfico mediante MaxMind GeoLite2 y cobertura de tests superior al 80%.

Tecnologías	Python 3.x · pandas · PyArrow · GeoIP2 · pytest
Almacenamiento	Apache Parquet (Star Schema)
Cobertura de tests	≥ 80 %
Última revisión	Mayo 2026

Tabla de Contenidos

1. Visión General del Sistema
2. Arquitectura y Modelo de Datos
 - 2.1 Estructura de directorios
 - 2.2 Star Schema — Modelo dimensional
3. Módulos del Sistema
 - 3.1 config.py — Configuración centralizada
 - 3.2 main.py — Orquestador del pipeline
 - 3.3 src/ReadProcess.py — Núcleo ETL
 - 3.4 src/localization.py — Enriquecimiento geográfico
 - 3.5 src/validate.py — Validación de calidad
 - 3.6 src/logger.py — Sistema de logging
 - 3.7 src/utils.py — Utilidades compartidas
4. Suite de Tests
5. Artefactos Generados
6. Patrones de Diseño y Convenciones
7. Ejecución y Despliegue

1. Visión General del Sistema

Portafolio Spotify es un pipeline ETL (Extract, Transform, Load) diseñado para procesar el historial de reproducción exportado por Spotify en formato JSON. El sistema transforma los datos crudos en un data warehouse dimensional, implementando un esquema en estrella persistido en Apache Parquet, óptimo para su consumo posterior en herramientas de Business Intelligence como Power BI.

Propiedad	Descripción
Idempotencia	El pipeline puede ejecutarse múltiples veces sin efectos secundarios.
Tolerancia a fallos	Errores parciales (ej. GeoLite2 no disponible) no abortan el proceso.
Multi-usuario	Variable de entorno SPOTIFY_USER_SUFFIX segrega datos por usuario.
Auditabilidad	Logging completo con timestamps en consola y archivo rotativo.
Escalabilidad	Arquitectura modular que permite agregar nuevas dimensiones sin refactoring mayor.

2. Arquitectura y Modelo de Datos

2.1 Estructura de Directorios

El repositorio sigue una separación clara entre código fuente, datos y artefactos generados. Los directorios de datos no están versionados; se crean en runtime.

Ruta	Contenido
src/	Módulos del pipeline: ETL, geolocalización, validación, logger, utils.
tests/	Suite pytest: unitarios, integración, fixtures y mocks.
data/raw{SUFIJO}/	JSONs de entrada exportados desde Spotify.
data/processed{SUFIJO}/	Archivos Parquet generados por el pipeline.
data/Databases/	Bases MaxMind GeoLite2 (.mmdb). No versionadas.
logs/	Archivos de log generados en runtime (pipeline_YYYYMMDD_HHMMSS.log).
docs/images/	Capturas de visualizaciones Power BI.
config.py	Fuente única de rutas y constantes globales.
main.py	Punto de entrada; orquesta todo el flujo.

2.2 Star Schema — Modelo Dimensional

El modelo de datos sigue el patrón star schema estándar: una tabla de hechos central con claves foráneas hacia cinco dimensiones normalizadas. Las claves surrogate son enteros secuenciales; el valor -1 representa la fila 'Unknown' en cada dimensión, garantizando integridad referencial ante datos nulos o desconocidos.

Tabla	Clave Natural	Atributos Principales
fact_table	event_id (PK)	date_id, track_id, episode_id, device_id, location_id, ms_played, skipped, shuffle, offline, incognito_mode
dim_date	YYYYMMDDHH	year, month, day, weekday, hour
dim_device	platform	device_type (mobile / desktop / web / other)
dim_track	spotify_track_uri	track_name, artist_name, album_name
dim_episode	spotify_episode_uri	episode_name, show_name
dim_location	ip_addr	conn_country + (post-enriquecimiento) city, region, isp, lat, lng

3. Módulos del Sistema

3.1 config.py — Configuración Centralizada

Único archivo de configuración importado por todos los módulos. Define rutas base, constantes de dominio y parámetros de logging. Al ser importado, crea los directorios necesarios (raw, processed, logs, databases) con `mkdir(parents=True, exist_ok=True)`, lo que implica efectos secundarios en la importación; en tests se parchean estas rutas.

Constante / Variable	Propósito
USER_SUFFIX	Sufijo desde env SPOTIFY_USER_SUFFIX. Permite multi-usuario sin cambios de código.
RAW_DATA_DIR / PROCESSED_DATA_DIR	Rutas de entrada y salida parametrizadas por sufijo de usuario.
CITY_DB / ASN_DB	Rutas a bases MaxMind GeoLite2-City.mmbd y GeoLite2-ASN.mmbd.
DIM_*_FILE / FACT_TABLE_FILE	Rutas absolutas a cada Parquet de salida.
UNKNOWN_VALUE / UNKNOWN_ID	Sentinelas para dimensiones y FKs ausentes ('Unknown' y -1).
AUDIOBOOK_COLUMNS_PATTERN	Subcadena 'audiobook'; columnas completamente nulas con este patrón se eliminan.

3.2 main.py — Orquestador del Pipeline

Punto de entrada CLI. Ejecuta las cuatro fases del pipeline en secuencia, con manejo diferenciado de errores: los fallos en ETL son fatales (`sys.exit 1`), mientras que los fallos en geolocalización o validación generan advertencias y permiten la continuación.

Fase	Módulo invocado	Comportamiento ante fallo
1. Pre-validación	src/utils.py	Fatal — <code>sys.exit(1)</code>
2. ETL	src/ReadProcess.main()	Fatal — <code>sys.exit(1)</code>
3. Geolocalización	src/localization.enrich_location_dimension()	No fatal — advertencia, continúa
4. Validación	src/validate.validate_data()	No fatal — advertencia, continúa

3.3 src/ReadProcess.py — Núcleo ETL

Módulo principal del pipeline (~400 líneas). Implementa las tres fases clásicas ETL más la creación del esquema dimensional.

Extract

`extract_json_files(path)` lista todos los archivos *.json en el directorio indicado, los lee con `pd.read_json` usando `convert_dates=['ts']` para parsear el timestamp, y los concatena en un único DataFrame. Lanza `FileNotFoundError` si el directorio está vacío.

Clean

`clean_data(df)` elimina columnas cuyo nombre contiene el patrón 'audiobook' si todas sus celdas son nulas, realiza `reset_index` y agrega `event_id` como clave primaria secuencial (0..N-1).

Transform — Creación de Dimensiones

Cada función `create_dim_*` sigue el mismo patrón: selección de atributos relevantes, deduplicación por clave natural, asignación de surrogate key entero y prepend de una fila 'Unknown' con `id=-1`. Para dimensiones `track` y `episode`, se filtra previamente por URI no nulo y se usa `pd.StringDtype()` para alinear tipos en los merges.

Load — Tabla de Hechos

`create_fact_table(df)` recalcula `date_id` desde el timestamp, realiza merges left con cada dimensión (solo clave natural + surrogate para no inflar columnas), rellena FK nulos con `UNKNOWN_ID=-1` y convierte columnas booleanas (`skipped`, `shuffle`, `offline`, `incognito_mode`) a tipo bool para compatibilidad con PyArrow/Parquet.

3.4 src/localization.py — Enriquecimiento Geográfico

Post-proceso sobre `dim_location.parquet` que agrega información geográfica mediante MaxMind GeoLite2. Opera sobre las IPs únicas del dataset, filtrando aquellas que no tienen representación geográfica válida antes de consultar las bases.

Filtrado de IPs

Tipo de IP	Acción
RFC 1918 (10.x, 172.16.x, 192.168.x)	Omitir — no tienen ubicación geográfica.
Loopback, link-local, multicast, unspecified	Omitir.
Placeholders (0.0.0.0, 'unknown', 'Unknown')	Omitir.
IPv6 privadas / loopback / link-local	Omitir.
IPs públicas válidas	Consultar GeoLite2-City y GeoLite2-ASN.

El resultado se une (`merge left`) con `dim_location` original, preservando `conn_country` (código ISO de Spotify) y agregando `country` (nombre largo de MaxMind), `city`, `region`, `isp`, `latitude` y `longitude`. IPs no encontradas en la base quedan con valores 'Unknown' y coordenadas (0.0, 0.0).

3.5 src/validate.py — Validación de Calidad

Módulo de inspección post-enriquecimiento (~100 líneas). Lee `dim_location_enriched.parquet` y registra estadísticas de cobertura: total de registros, conteos de valores no-Unknown por columna geográfica, top-10 ciudades y top-10 ISPs. Cada validación está envuelta en `try-except` independiente para garantizar que un fallo parcial no interrumpa el reporte.

3.6 src/logger.py — Sistema de Logging

Provee el logger singleton 'spotify_pipeline' con dos handlers: StreamHandler a stdout para visibilidad en terminal, y FileHandler a logs/pipeline_YYYYMMDD_HHMMSS.log con encoding UTF-8. El formato estándar es: YYYY-MM-DD HH:MM:SS | LEVEL | módulo | mensaje. Si la creación del archivo de log falla, el sistema continúa operando solo con la consola.

3.7 src/utlis.py — Utilidades Compartidas

Función	Contrato
validate_input_file(path, file_type)	Verifica existencia y que sea archivo (no directorio). Retorna bool.
validate_input_directory(path, must_contain)	Verifica directorio; si must_contain='*.json', exige al menos un match con glob.
ensure_output_directory(output_dir)	mkdir(parents=True, exist_ok=True). Captura excepciones; retorna bool.

4. Suite de Tests

El proyecto cuenta con una suite pytest con cobertura superior al 80% del código fuente, organizada en tests unitarios y de integración. La configuración en `pytest.ini` habilita cobertura con reporte HTML (`htmlcov/`) y marcadores estrictos para filtrado selectivo.

Archivo	Módulo Testeado	Tipo
<code>test_read_process.py</code>	<code>src/ReadProcess.py</code>	Unitario + Integración
<code>test_localization.py</code>	<code>src/localization.py</code>	Unitario
<code>test_pipeline.py</code>	<code>main.py</code> — flujo completo	Integración end-to-end
<code>test_utils.py</code>	<code>src/utils.py</code>	Unitario
<code>test_validate.py</code>	<code>src/validate.py</code>	Unitario

Fixtures y Mocks Destacados

`confest.py` provee DataFrames mínimos con IPs de documentación RFC 5737 (evitan resolución geográfica real), una estructura de directorios temporales mediante `tmp_path` de `pytest` y un mock de `geoip2.database.Reader` con atributos primitivos explícitos (strings y floats), necesario para evitar tipos no serializables por `PyArrow` al guardar Parquet.

5. Artefactos Generados

El pipeline produce siete archivos Parquet en el directorio `processed{SUFIJO}/`. Todos se generan sin índice (`index=False`) y son compatibles con Power BI y otras herramientas BI vía conector Parquet.

Archivo	Origen	Descripción
<code>dim_date.parquet</code>	<code>ReadProcess</code>	Dimensión temporal con granularidad horaria (YYYYMMDDHH).
<code>dim_device.parquet</code>	<code>ReadProcess</code>	Plataformas clasificadas en mobile, desktop, web, other.
<code>dim_track.parquet</code>	<code>ReadProcess</code>	Catálogo de canciones con URI, artista y álbum.
<code>dim_episode.parquet</code>	<code>ReadProcess</code>	Catálogo de episodios/podcasts.
<code>dim_location.parquet</code>	<code>ReadProcess</code>	IPs únicas con código ISO de país (Spotify).
<code>fact_table.parquet</code>	<code>ReadProcess</code>	Tabla de hechos con FKs, <code>ms_played</code> y flags booleanos.
<code>dim_location_enriched.parquet</code>	<code>localization</code>	<code>dim_location</code> + city, region, country, isp, lat, lng (MaxMind).

6. Patrones de Diseño y Convenciones

Patrón	Aplicación en el Proyecto
Star Schema	Tabla de hechos central con dimensiones normalizadas; optimizado para queries analíticas.
ETL Pipeline	Separación explícita en fases Extract, Transform y Load con responsabilidades claras.
Singleton	Logger global y config.py como fuente única de configuración.
Factory	Funciones create_dim_*() como factories de dimensiones con contrato uniforme.
Strategy	classify_device() como estrategia intercambiable de clasificación de plataformas.

Convenciones de Código

Nomenclatura: snake_case para funciones y variables, UPPER_SNAKE_CASE para constantes, prefijo _ para funciones privadas de módulo (ej. _skip_geolocation_ip). Todos los parámetros y valores de retorno cuentan con type hints. Docstrings en formato Google/NumPy con secciones Args, Returns y Raises. Los errores no críticos se capturan con excepciones específicas y se registran con exc_info=True para incluir el stack trace completo en el log.

7. Ejecución y Despliegue

Requisitos

Python 3.x con las dependencias declaradas en requirements.txt: pandas, numpy, pyarrow, geoip2, pytest, pytest-mock, pytest-cov y python-dotenv. Para el enriquecimiento geográfico se requieren las bases GeoLite2-City.mmdb y GeoLite2-ASN.mmdb de MaxMind (no incluidas en el repositorio).

Instalación

```
pip install -r requirements.txt
```

Ejecución del Pipeline Completo

```
# Usuario por defecto (data/raw, data/processed)
python main.py

# Usuario alternativo (data/rawPedro, data/processedPedro)
SPOTIFY_USER_SUFFIX=Pedro python main.py
```

Ejecución por Fases

```
python -m src.ReadProcess      # Solo ETL
python -m src.localization     # Solo enriquecimiento geo
python -m src.validate         # Solo validación
```

Tests

```
pytest                        # Suite completa
pytest -m unit                # Solo unitarios
pytest -m integration         # Solo integración
pytest --cov=src --cov-report=html # Con reporte de cobertura HTML
```